

11강 강의 대본

Gemini 생태계 마스터 — AI에게 "환경을 깔아라"고 시키는 법

강의 시간	40분 (빠른 템포 발화 기준)
대상	디스플레이·반도체 현장 엔지니어
촬영 구성	교안 화면 녹화 + 강사 캠 PIP
교안 URL	https://heekeunlee.github.io/lecture11/

대본 읽는 법

【지문】→ 카메라·화면 동작, 강사 행동 지시

(괄호 안) → 말투·속도·강조 지시

굵게 → 강조하여 또렷하게 발음

[잠깐] → 1~2초 호흡 멈춤

▶ **AI에게 이렇게 시킵니다** → 학생이 AI에 입력하는 지시문(프롬프트)

AI가 만들어 준 결과 → AI가 자동 생성한 코드/출력 (사람은 검증만)

강의 흐름 요약

시간	교안 섹션	핵심 내용
0:00	헤더 + 00. Learning Goal	인트로 — "오늘부터 코드는 AI가 짭니다"
3:00	01. Gemini 생태계 개요	5개 도구·월 \$20 가성비
8:00	02. AI 작업 환경 선택	Cursor / Claude Code / Gemini CLI
13:00	03. API Key 발급 (직접)	AI가 못 하는 3분짜리 사람 작업
18:00	04. 첫 바이브 코딩 시연	AI에게 "환경 깔고 Hello 보내라"
26:00	05. NotebookLM 지식 베이스	논문 업로드 → 출처 기반 답변
32:00	06. Firebase 프로젝트	AI에게 "init 도와줘"
36:00	07. AI 결과 검증 게이트	5개 체크포인트
38:30	08. 마무리 & 12강 예고	RAG 챗봇으로

SECTION 00 · 인트로 & 학습목표 0:00 ~ 3:00 / 3분

교안: 헤더 + 00. Learning Goal

【지문】헤더 폴스크린. 강사 캠 PIP. 1강의 "코드 한 줄도 직접 안 썼어요" 자막을 1초간 회상.

안녕하세요, 11강에 오신 걸 환영합니다.

(차분히) 1강에서 약속드렸죠? "코드 한 줄도 직접 안 쓴다"고. [잠깐]

오늘부터 — 그 약속을 — **시연**합니다. 화면에 코드가 나오긴 합니다. — 그런데 그 코드는 — **AI가 만든 결과**예요. 여러분이 타이핑하는 건 — **AI에게 보내는 지시문**입니다.

"오늘 여러분이 입력할 단어 수와 —
오늘 화면에 등장할 코드 줄 수를 — 끝에서 비교해 봅시다."

【지문】학습목표 카드 3개 (Bot / Key / Cloud)

오늘의 목표 세 개.

첫째, Gemini 생태계 5개 도구를 — **한눈에** 파악합니다.

둘째, AI 작업 환경(Cursor·Claude Code·Gemini CLI)을 **하나** 선택합니다.

셋째, AI에게 지시해서 — **Hello Gemini** 호출까지 완성합니다. 코드는 AI가 씁니다.

SECTION 01 · Gemini 생태계 개요 — 5개 도구·월 \$20 3:00 ~ 8:00 / 5분

교안: 01. Gemini Ecosystem

(빠르게) Gemini는 챗봇이 아닙니다. 다섯 개 도구로 묶인 **생태계**입니다. [잠깐]

도구	가격	쓰임	오늘 다룸
Gemini Pro (chat)	\$20/월	긴 문서·논문 분석, Deep Research	스치듯
Google AI Studio	무료	API Key 발급·프롬프트 테스트	직접
NotebookLM	무료	출처 기반 지식 베이스 (RAG 원본)	직접
Firebase	무료(Spark)	Hosting·DB·Auth — 15강 메인	init만
Telegram Bot API	무료	알림 자동화 — 17강 메인	예고만

"월 \$20 한 장으로 — API·지식 베이스·배포·알림이 — 모두 무료로 따라옵니다.
"엔지니어가 코드를 안 쓰는 시대"의 — 가장 비싼 가성비입니다."

SECTION 02 · AI 작업 환경 선택 — Cursor · Claude Code · Gemini CLI 8:00 ~

13:00 / 5분

교안: 02. Pick Your AI Workbench

【지문】세 IDE 스크린샷이 카드로 나란히. 강사는 손가락으로 한 번씩 가리키며 비교.

(설명조) 여기서 오늘의 핵심이 정해집니다. — 여러분이 일할 "바이브 코딩 작업대". [잠깐]

도구	특징	추천 사용자
Cursor	VS Code 포크 · 채팅창 + 코드 인라인 편집	처음 시작 · 화면이 익숙한 분
Claude Code (CLI)	터미널에서 자연어로 프로젝트 전체 조작	터미널이 편한 분 · 자동화 좋아하는 분
Gemini CLI	Google 공식, Gemini 모델 최적	Gemini 100만 토큰을 풀로 쓰고 싶은 분

"오늘은 Cursor로 시연합니다.
여러분은 — 본인 환경 하나 골라서 — 같은 지시문을 따라 보내면 됩니다."

(중요) AI에게 보내는 지시문은 — 도구가 달라도 거의 똑같이 통합니다. — 이게 바이브 코딩의 매력입니다.
[잠깐]

SECTION 03 · API Key 발급 — AI가 못 하는 사람 작업 3분 13:00 ~ 18:00 / 5분

교안: 03. Get the API Key (you)

【지문】브라우저로 전환. aistudio.google.com 로딩 화면 → 'Get API Key' 클릭.

(차분히) API Key 발급은 — 유일하게 AI가 못 하는 단계입니다. 본인 Google 계정으로 — 직접 로그인해서 받으셔야 합니다. [잠깐]

(빠르게 시연 순서) 1) aistudio.google.com 접속. 2) Get API Key 클릭. 3) Create API Key in new project. 4) 생성된 키 — 한 번만 보입니다. 바로 복사. [잠깐]

이걸 — 메모장에 잠깐 두세요. 다음 섹션에서 — AI한테 .env 파일 만들라고 시킵니다.

"오늘 여러분이 타이핑하는 3분은 — 이 3분이 — 거의 전부입니다.
나머지 37분은 — AI에게 시키는 시간입니다."

SECTION 04 · 첫 바이브 코딩 시연 — "환경 깔고 Hello 보내라" 18:00 ~ 26:00 / 8분

교안: 04. First Vibe Coding Demo

【지문】Cursor를 폴스크린으로. 채팅창에 강사가 한국어를 입력하는 동작을 천천히 보여준다.

(에너지 살짝 업) 자, 진짜 시작합니다. 새 폴더 만들고 — Cursor를 엽니다. 그리고 — 채팅창에 — 이걸 그대로 보냅니다. [잠깐]

▶ AI에게 이렇게 시킵니다 (Cursor 채팅창에 그대로 붙여넣기)

프로젝트 폴더에 Gemini API를 부르는 가장 작은 Python 환경을 만들어줘.

조건:

- 1) google-generativeai, python-dotenv 설치 명령을 README.md에 안내
 - 2) .env 파일을 만들고, 안에는 GEMINI_API_KEY=여기에키만 들어가도록
 - 3) .gitignore에 .env를 추가
 - 4) hello.py 파일에 5줄 이내로 Hello Gemini 호출 코드 작성
 - 5) 실행하면 응답 텍스트를 그대로 print 하도록
- 파일 4개(README.md, .env, .gitignore, hello.py)를 모두 만들어줘.

(잠깐, 강조) 보세요. 이게 오늘 여러분이 칠 거의 전부입니다. — 6줄짜리 지시문. [잠깐]

Cursor 또는 Claude Code는 — 이 지시문 한 방에 — 파일 네 개를 — 자동으로 생성합니다. 여러분은 수락(accept) 버튼만 누르면 됩니다.

AI가 만들어 준 파일 (사람은 검증만)

```
# README.md (AI ■■■ ■■■ - ■■■ ■■■)
pip install google-generativeai python-dotenv
python hello.py
# hello.py (AI ■■■ ■■■)
import google.generativeai as genai, os
from dotenv import load_dotenv; load_dotenv()
genai.configure(api_key=os.getenv('GEMINI_API_KEY'))
model = genai.GenerativeModel('gemini-1.5-pro')
print(model.generate_content('Hello, Gemini!').text)
```

(설명조 · 엔지니어가 할 일) 우리가 할 일은 — 이제 두 가지입니다. [잠깐]

하나 — 13분 섹션에서 발급받은 키를 — .env의 "여기에키" 자리에 — 붙여넣습니다.

둘 — 터미널에서 `pip install ...` 한 번, `python hello.py` 한 번. — 응답이 뜨면 — 성공입니다.

(잠깐) 그게 끝입니다. 여러분이 친 코드 줄 수는 0. 친 한국어 줄 수만 — 위에 보이는 6줄.

실수했을 때도 — AI에게 시킵니다

▶ AI에게 디버깅도 시킵니다

방금 만든 `hello.py`를 실행했더니 아래 에러가 떴어. 원인과 해결 방법을 알려줘.
그리고 `hello.py`를 안전하게 고쳐줘.

(여기에 에러 메시지 그대로 복사 붙여넣기)

(중요) 디버깅도 AI한테 시켜요. 에러 메시지를 — 그대로 채팅창에 던지면 됩니다. [잠깐]

이게 — 1장에서 말한 "감이 아니라 의도"의 — 디버깅 버전입니다.

SECTION 05 · NotebookLM 지식 베이스 — 12강의 씨앗 26:00 ~ 32:00 / 6분

교안: 05. NotebookLM Seeds

【지문】notebooklm.google.com으로 전환. PDF를 끌어다 놓는 동작.

(차분히) NotebookLM은 — 코드를 짤 일이 거의 없어요. GUI예요. [잠깐]

여러분이 할 일 — PDF나 웹링크를 — 끌어다 놓는다. 끝. 그러면 AI가 — 그 자료만 보고 답합니다.

▶ NotebookLM 채팅창에 이렇게 묻습니다

이 노트북에 업로드된 자료들의 핵심 기여를 3가지로 정리해줘.
각 기여 옆에 출처 페이지 번호를 명시해줘.
답변은 한국어 5문장 이내로.

(설명) 답변 끝에 — [출처: 3페이지, Figure 2] 같은 라벨이 자동으로 붙습니다. [잠깐]

이게 — "할루시네이션 없는 답변"의 — 메커니즘이고, 12강에서 만들 챗봇의 원형입니다.

SECTION 06 · Firebase 프로젝트 — AI에게 "init 도와줘" 32:00 ~ 36:00 / 4분

교안: 06. Firebase Init by AI

【지문】Firebase Console 'Add project' 클릭하는 모습. 콘솔 작업과 터미널 작업이 동시에 보이도록 분할.

(차분히) Firebase는 — 콘솔에서 프로젝트 생성만 사람이 합니다. (Add project → 이름 → Create.) 그다음부터는 — 또 — AI에게 시킵니다. [잠깐]

▶ AI에게 Firebase init을 맡깁니다

이 프로젝트 폴더에 Firebase Hosting을 초기화해줘.

조건:

- firebase-tools를 설치하는 명령어를 README.md에 추가
- firebase.json은 SPA 단일 페이지 앱 설정
- public 폴더에 임시 index.html("Hello, Firebase!") 한 개 생성
- .gitignore에 .firebase, *.log를 추가
- 실행 순서를 README.md에 step-by-step으로 적어줘

(중요) AI가 만든 README의 — "firebase login" 단계는 — 사람이 직접 해야 됩니다. 본인 계정이니깐요.

[잠깐]

그것만 빼고 — 나머지는 — AI가 만든 파일 그대로 — `firebase deploy` 한 번이면 — 배포 끝.

(예고) 15강에서 — 이 환경에 — 우리가 만든 시뮬레이터를 — 엽니다.

SECTION 07 · AI 결과 검증 게이트 — 5개 체크포인트 36:00 ~ 38:30 / 2.5분

교안: 07. Quality Gate

(체크 손동작) 바이브 코딩에서 — "검수"가 곧 엔지니어의 본업입니다. [잠깐]

AI가 만든 결과를 — 5개로 확인합니다.

hello.py가 실행되어 — Gemini 응답이 콘솔에 출력되었는가?

.env가 .gitignore에 포함되어 `git status`에 노출되지 않는가?

API Key가 코드 본문에 직접 적혀 있지 않은가? (AI가 가끔 깜빡합니다 — 사람 검수 필수)

NotebookLM 답변에 — 출처 라벨이 붙어 있는가?

Firebase 콘솔에 — 본인의 프로젝트가 생성되어 있는가?

(강조) 이 다섯 개에 — 다 체크가 되면 — 오늘 11강은 — 완벽 통과입니다. [잠깐]

체크 안 된 항목 — 그 자체로 다음 AI 지시문이 됩니다. 예: "3번이 안 됐어. API Key가 코드에 박혀 있어. 고쳐줘."

SECTION 08 · 마무리 & 12강 예고 38:30 ~ 40:00 / 1.5분

교안: 08. Wrap-up & Bridge

"여러분이 오늘 타이핑한 한국어 줄 수와 —

화면에 등장한 코드 줄 수를 — 세어 보세요.

— 코드는 — 여러분이 짜지 않았습니다."

다음 12강 — 오늘 만든 환경 위에서 — 디스플레이 전공 챗봇을 만듭니다. 코드는 — 역시 — AI가 짭니다.

[잠깐]

(밝게) 11강 수고하셨습니다!

부록 A · 강의 흐름 타임라인

구간	시간	교안 섹션	핵심 멘트
인트로	0:00~3:00	헤더 + 00	"오늘 코드는 AI가 짭니다."
생태계	3:00~8:00	01	"월 \$20에 5개가 따라온다."
작업대	8:00~13:00	02	"Cursor / Claude Code / Gemini CLI."
API Key	13:00~18:00	03	"AI가 못 하는 3분."
바이브 시연	18:00~26:00	04	"한국어 6줄 → 파일 4개."
NotebookLM	26:00~32:00	05	"드래그앤드롭 + 출처."
Firebase	32:00~36:00	06	"콘솔 1회, AI 1회."
검수	36:00~38:30	07	"5개 체크 = 사람의 본업."
마무리	38:30~40:00	08	"코드는 짜지 않았다."

부록 B · 강사 유의사항

항목	내용
강의 톤	기술 강의이지만 — "AI가 짭다" 메시지가 일관되게 흐르도록. 코드 줄은 "이건 결과물" 자막 유지.
실수 방지	AI가 만든 코드에 API Key가 박혀 나오면 — 그 자리에서 사람이 빼는 시범을 1회 보여줄 것.
화면 전환	Cursor ↔ 브라우저(AI Studio·NotebookLM·Firebase) ↔ 터미널 — 3개 전환이 잦음. 매번 안내 멘트.
실습 침묵	AI 응답 대기 5~7초 — "기다리는 시간이 본업 아닙니다" 한마디로 분위기 환기.
강조 반복	"여러분이 친 한국어 vs 화면의 코드 줄 수" — 본 강의에서 2회 비교.
에너지	마지막 "코드는 짜지 않았다" 마무리는 1강 톤을 회수. 1초 정적 후 페이드아웃.

© 2026 LettUin Edu — Vibe Coding for Fine Tech Engineering | 강의 교안: <https://heekeunlee.github.io/lecture11/>